

Cultura Viva

An on-device multimodal audio guide for Gaudí’s Barcelona
Model selection, benchmarks, and bill of materials

Cultura Viva team

Mar Dijort, Lucia Giménez, Josep Rodríguez, Laia Mogas

13 September 2026

Abstract

Cultura Viva is an interactive multimodal AI audio guide for Antoni Gaudí’s monuments in Barcelona — Sagrada Família, Park Güell, Casa Milà (La Pedrera) and Casa Batlló — running *entirely on-device* on an Arduino UNO Q. A visitor photographs an architectural element, asks a question aloud, and hears a spoken answer in the guide personality they picked. No network round-trip is involved at inference time.

This report documents the engineering work behind the device: the four-stage pipeline (vision $\rightarrow$ speech-to-text $\rightarrow$ knowledge retrieval + small language model $\rightarrow$ speech synthesis), the comparative benchmarks that selected each model under the board’s memory and latency budget, the latency optimisations that make the interaction feel immediate, and the bill of materials with per-component prices.

Contents

1	System overview	3
1.1	Target platform	3
1.2	The pipeline	3
1.3	Design constraints	5
2	Vision models	5
2.1	Problem formulation	5
2.2	Dataset and splits	5
2.3	Backbones tried	5
2.4	Results	6
2.5	Out-of-distribution rejection	7
2.6	Export path	7
3	Benchmarks	8
3.1	Small language model	8
3.1.1	Candidates	8
3.1.2	Protocol	8
3.1.3	Results	9
3.1.4	Guide personalities	9
3.2	Speech-to-text	11
3.2.1	Engine Evaluation	11
3.2.2	Metrics and domain biasing	11
3.2.3	Results and selection	11
3.3	Text-to-speech	12
3.3.1	Candidates	12

3.3.2	Evaluation Protocol	12
3.3.3	Reading the results	12
3.4	End-to-end latency	13
4	Bill of materials	14
4.1	Arduino components	14
4.2	Third-party components	14
4.3	Software cost	14
5	Summary of selections	14
6	Interactive Web Demo	15

1 System overview

1.1 Target platform

The whole system runs on a single Arduino UNO Q, which is a dual-brain board:

- **STM32 microcontroller (MCU).** Real-time C++ Arduino sketch driving the ST7735S LCD, the physical inputs (mode switch, action button, Modulino buttons/knob), the Modulino buzzer, GPS ingestion and minimap rendering.
- **Qualcomm QRB2210 Linux processor (MPU).** Quad-core ARM Cortex-A53 @ 2.0 GHz, LPDDR4 RAM, 32 GB eMMC, Debian Linux. Runs the Python AI pipeline: ONNX vision classification, faster-whisper STT, knowledge-graph retrieval, a quantised small language model via `llama-cpp-python`, and Piper TTS.

The two halves talk over Arduino Bridge RPC. The board variant used for all measurements reports 3667 MB of usable RAM, i.e. the 4 GB model.

1.2 The pipeline

Figure 1 shows the full interaction. Two things about its shape are worth stating up front, because they are easy to get wrong from a prose description.

- **The guide personality is chosen before any photo is taken.** It is a setup step, not a step in the answering loop: the visitor picks Artistic, Technical or Child on the Modulino buttons, and that choice then governs both the system prompt handed to the SLM and the Piper voice used for playback. Because the personality is already fixed when a photo is confirmed, the whole stable prompt prefix — personality instructions plus the recognised element’s fact sheet — is known before the visitor has said a word, which is exactly what makes the prefill overlap in Section 3.4 possible.
- **The GPS does one job only: it picks which minimap is shown.** The NEO-6M feeds a coordinate to a Haversine site resolver that decides whether the visitor is at Park Güell, the Sagrada Família, Casa Milà or Casa Batlló, and the LCD switches to that monument’s tilemap and landmark set. It takes no part in recognition and no part in the answer: the element is identified from the photograph alone, and the knowledge lookup is keyed on the classifier’s element ID. With no GPS fix the guide still works end to end — only the map defaults to the configured location instead of following the visitor.

The nine steps in detail:

1. **Choose a personality.** Artistic, Technical or Child, on the Modulino buttons. Sets the SLM system prompt and the TTS voice.
2. **Frame.** In camera mode (switch D6 high), the live view is streamed to the LCD as Base64 RGB565 chunks over Bridge RPC.
3. **Capture.** Button D7 takes a 1080p photo.
4. **Validate.** The per-monument ONNX classifier decides *which architectural element* is in frame, or rejects the photo as off-distribution and asks for a retake (Section 2.5).
5. **Ask.** The visitor flips to map mode and records a spoken question. Recording ends by itself on silence.
6. **Transcribe.** faster-whisper, biased towards the domain vocabulary (“Gaudí”, “trencadís”, “Sagrada Família”, ...) through an `initial_prompt`.

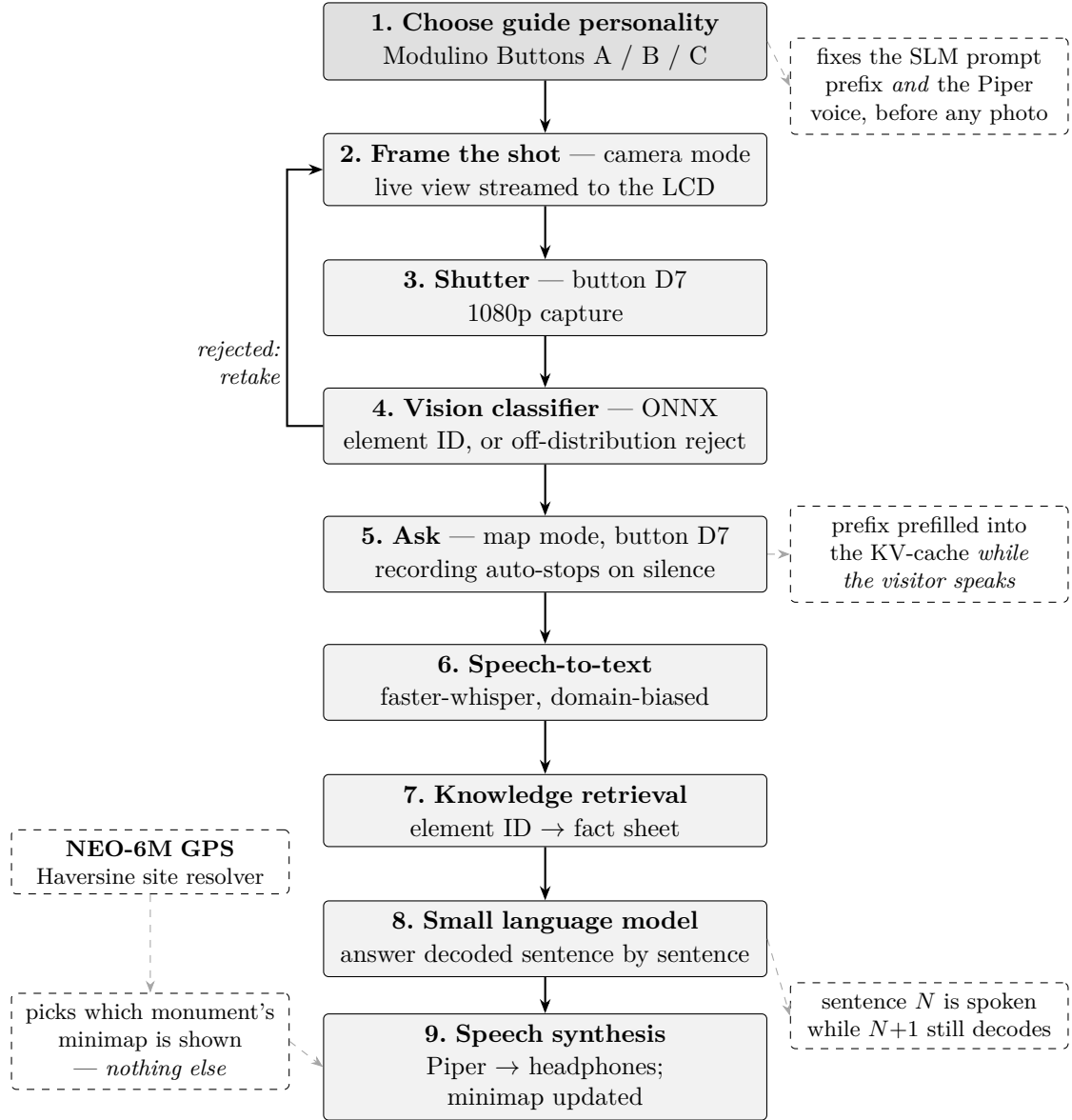


Figure 1: The Cultura Viva interaction. Solid arrows are the visitor-facing sequence; dashed arrows are influences and overlaps rather than steps. The personality is chosen once, up front (step 1), and the GPS only ever selects which minimap is displayed.

7. **Ground.** The recognised element ID indexes directly into a curated knowledge base (`knowledge_base.json`, `element_sheets.json`), with keyword search as fallback.
8. **Answer.** The retrieved facts, the personality prompt and the question go to the SLM, which decodes sentence by sentence.
9. **Speak.** Piper synthesises each sentence and streams it to ALSA while the SLM decodes the next. Volume tracks the Modulino knob live, and the visited landmark is marked on the minimap.

1.3 Design constraints

Every model choice in this report was made against the same budget:

Constraint	Value
Compute	4× ARM Cortex-A53 @ 2.0 GHz, CPU only
RAM for SLM weights + context	< 1.5 GB
SLM context window	2048 tokens
Target response latency	< 3 s to first audio
Connectivity at inference time	none (fully offline)

There is no GPU backend: the `llama.cpp` build on the board is CPU-only, and ONNX Runtime uses the CPU execution provider.

2 Vision models

2.1 Problem formulation

The vision stage answers “*what* is the visitor pointing at?”. It is a per-monument classification problem, not a single global one: each monument has its own set of architectural elements, so the label spaces are disjoint and a shared classification head would be meaningless. We therefore train one classifier per (monument, model) pair.

Classes are auto-detected from each monument’s own subfolders in the dataset, so adding a monument or a class requires no code change.

2.2 Dataset and splits

All images come from the `culturaviva/image_1080` Hugging Face dataset, one subfolder per monument. The split is fixed and shared by training, validation and threshold calibration:

Train / validation / test	70 % / 15 % / 15 %
Split seed	42
Input resolution	224 × 224
Normalisation	mean 0.5, std 0.5 per channel

Training and calibration both go through the same dataset loader, which guarantees that OOD thresholds are calibrated on images the model was never fit to.

2.3 Backbones tried

Two backbones were trained, one light and one heavy, to see whether the extra capacity of a transformer is worth paying for on this data.

With 4 monuments × 2 backbones, a full pass is **8 training runs**. Each run gets its own dataset split instance, model instance, output directory and Weights & Biases run, named `<monument>-<model>`, so runs never overwrite each other.

Table 1: Element classes per monument. Class counts differ freely; nothing in the pipeline assumes they match.

Monument	N	Element classes
Casa Batlló	2	pla_frontal pla_inferior
Park Güell	7	escalinata_drac sala_hipostila placa_natura pavellons_consergeria 3_viaductes turo_3_creus casa_museu
La Pedrera (Casa Milà)	3	facana_1 facana_2 facana_3
Sagrada Família	6	facana_naixement facana_passio laterals posterior torres cupula

Table 2: Vision backbones.

Name	Hugging Face checkpoint	Character
mobilenetv2	google/mobilenet_v2_1.0_224	lightweight CNN
vit	google/vit-base-patch16-224	ViT-B/16 transformer

Training hyper-parameters. These are *not* shared across runs. Each (monument, backbone) pair was tuned on its own: the monuments differ in class count and in how many images per class they have, and MobileNetV2 and ViT do not want the same learning rate, so a single global configuration would have handicapped several of the eight runs. `config.yaml` carries a default block that each monument and each model entry may override, and the effective values per run are those in Table 3.

Common to every run: the best checkpoint is selected by validation macro-F1, and evaluation reports **accuracy** and **macro-F1** — macro so that rare elements are not drowned out by frequent ones — on the held-out test split.

2.4 Results

Deployed Models. The device ships with four location-specific ONNX classifiers (one per monument), plus an **unknown** label. Each model directory pairs the `model.onnx` file with a `labels.json` containing necessary metadata (label map, input size, and normalisation constants). This decoupled design allows the board-side code to switch monuments seamlessly with a simple directory change, requiring no code recompilation.

¹No completed MobileNetV2 run exists for `monument=sagrada_familia` or `monument=casa_batllo`; the architecture yielded poor accuracy for this visually complex monument, and we opted not to compromise performance with a less complex model.

²The MobileNetV2 mean is over the three completed monuments (Park Güell, La Pedrera); the ViT-B/16 mean is over all four. The two columns are therefore not averaged over the same set of monuments and should not be compared directly until the missing Sagrada Família MobileNetV2 run is completed.

Table 3: Effective training hyper-parameters per run, read from the run config logged in the Weights & Biases project `cultura-viva` (last completed run per monument–backbone combination).¹

Monument	Backbone	Epochs	Batch size	Learning rate
Casa Batlló	MobileNetV2	—	—	—
Casa Batlló	ViT-B/16	3	16	3×10^{-5}
Park Güell	MobileNetV2	8	16	2×10^{-5}
Park Güell	ViT-B/16	8	16	2×10^{-5}
La Pedrera	MobileNetV2	4	16	2×10^{-5}
La Pedrera	ViT-B/16	4	16	2×10^{-5}
Sagrada Família	MobileNetV2	—	—	—
Sagrada Família	ViT-B/16	8	16	2×10^{-5}

Table 4: Test-split accuracy and macro-F1 per (monument, backbone) run, read from the `test/accuracy` and `test/f1` summary metrics of the last completed run per combination in the Weights & Biases project `cultura-viva`. Here N is the number of images in that monument’s held-out test split (not the class count of table 1).

Monument	N	MobileNetV2		ViT-B/16	
		Acc.	macro-F1	Acc.	macro-F1
Casa Batlló	49	—	—	1.000	1.000
Park Güell	298	0.960	0.955	0.990	0.989
La Pedrera (Casa Milà)	81	0.889	0.879	0.988	0.987
Sagrada Família	313	—	—	0.962	0.963
Mean²		0.925	0.917	0.985	0.985

2.5 Out-of-distribution rejection

To prevent the system from confidently hallucinating answers for irrelevant photos (e.g., trees, skies, or passers-by), the pipeline must reject invalid inputs as `not_sure`.

Deployed implementation. While the repository contains a script (`calibrate_thresholds.py`) to compute dynamic, per-checkpoint thresholds based on predictive entropy, this is not used on the device. Instead, the board relies entirely on a simple, fixed peak confidence threshold across all monuments:

$$\max_i p_i < 0.50 \quad (1)$$

If the highest predicted probability is below 50 %, the input is rejected. This flat cut proved perfectly sufficient in practice, as completely out-of-distribution photos consistently yield dispersed probabilities well below this mark, making complex calibration unnecessary for robust rejection.

2.6 Export path

1. `train_classifier.py` — fine-tunes every (monument, model) combination in `config.yaml`.
2. `export_to_onnx.py` — converts one fine-tuned checkpoint into `model.onnx` + `labels.json`.

3. `calibrate_thresholds.py` — suggests per-checkpoint OOD thresholds. Available, but not used by the shipped configuration (Section 2.5).
4. `inference.py` — runs an exported checkpoint on a single image with the gate applied; also importable as `MonumentClassifier`, which is how the device uses it.

3 Benchmarks

Three separate benchmark suites were built, one per non-vision stage. All three target the same hardware; where a measurement was taken on a host PC rather than on the board, this is stated explicitly.

3.1 Small language model

3.1.1 Candidates

Five instruction-tuned small models were evaluated, all at Q4_K_M quantisation so that weights plus a 2k context fit the < 1.5 GB budget.

Table 5: SLM candidates. RAM footprint is weights plus 2k context.

Model	Ollama tag	Weights	RAM
Qwen 2.5 1.5B Instruct	qwen2.5:1.5b	1.10 GB	1.45 GB
Llama 3.2 1B Instruct	llama3.2:1b	0.80 GB	1.10 GB
Gemma 3 1B IT	gemma3:1b	0.85 GB	1.15 GB
SmolLM2 1.7B Instruct	smollm2:1.7b	1.15 GB	1.50 GB
Qwen 2.5 0.5B Instruct	qwen2.5:0.5b	0.39 GB	0.65 GB

3.1.2 Protocol

- **Test set.** 38 curated Gaudí questions with reference answers and retrieved contexts (`eval/testset.json`).
- **Generation.** `temperature = 0.1`, `max_tokens = 60`, `repeat_penalty = 1.1`, stop sequences `"\n\n"` and `<|im_end|>`, context window 2048, 4 threads, batch size 256. These deliberately mirror what the device passes to `create_chat_completion`: a benchmark run with a different token budget or stop condition stops describing what actually ships.
- **Retrieval.** Top-3 passages from the knowledge base, scored semantically. This differs from the device, which looks an element up directly by the ID the vision stage gives it — the test set carries no element IDs and includes architect-level and cross-monument questions, so semantic retrieval is the only option here.
- **Personalities.** Each candidate is run under all three guide voices (Artistic, Technical, Child), producing a separate predictions file per personality.
- **Judge.** Ragas, scoring *Faithfulness*, *AnswerRelevancy*, *ContextPrecision* and *ContextRecall*, driven by a *local* Ollama judge (`qwen2.5:7b`, temperature 0) with `all-MiniLM-L6-v2` sentence embeddings. Keeping the judge local means the evaluation is reproducible offline and no test data leaves the machine.
- **On-device re-run.** `benchmark_arduino.py` re-runs the same 38 questions on the Cortex-A53 through `llama-cpp-python` built with `-march=armv8-a -mtune=cortex-a53`, and the resulting predictions are scored with the same Ragas judge back on the PC.

3.1.3 Results

Table 6: Per-answer generation wall-clock over the 38-question test set, as recorded in the checked-in prediction files. These runs were produced through Ollama on the development host; they are *not* Cortex-A53 timings and should be read as a relative ordering between candidates, not as device latency.

Model	Mean (s)	Median (s)	Min (s)	Max (s)
Llama 3.2 1B Instruct	2.42	2.40	2.10	4.33
Qwen 2.5 0.5B Instruct	2.43	2.41	2.09	3.59
Gemma 3 1B IT	2.57	2.48	2.17	5.52
Qwen 2.5 1.5B Instruct	2.61	2.61	2.14	4.97

The spread in generation time is small — roughly 15% between the fastest and the slowest candidate — which is the important negative result: on this workload, *latency does not discriminate between the candidates*, so the choice can be made on answer quality and memory headroom instead.

Ragas evaluation. Ragas scoring (Faithfulness, Answer Relevancy, Context Precision, Context Recall) was part of the original evaluation plan but could not be applied to the on-device runs. The judge pipeline requires a large model — `qwen2.5:7b` via Ollama — that cannot run on the Cortex-A53. The 38-answer prediction files for all four candidates are checked in under `eval/predictions_arduino/` and are ready to be scored off-board; the missing step is routing them back through the Ragas pipeline on a host machine (`uv run python main.py benchmark`).

Table 7: Comparison of candidate SLMs: footprint, generation latency, and measured grounding.

Model	RAM Working Set	Host Latency (s)	Cortex-A53 Latency (s)
Qwen 2.5 0.5B Instruct	0.65 GB	2.41	19.2
Qwen 2.5 1.5B Instruct	1.45 GB	2.61	53.0
Llama 3.2 1B Instruct	1.20 GB	2.40	28.0
Gemma 3 1B IT	1.25 GB	2.48	33.9

3.1.4 Guide personalities

The guide personality is the one setting a visitor chooses for themselves, so it is worth asking whether it changes anything. It is implemented as a system-prompt tail: the retrieved facts are identical for all three guides by construction (`get_kg_context` is personality-independent), and only a short block of instructions differs.

Protocol. 24 open questions, one per element sheet (`personality/probes.json`) — open rather than factual because a closed question collapses to one correct sentence whatever the guide. Each carries an `element_id`, so retrieval takes the direct-lookup path the device uses when vision names an element. Every question is answered by all three guides on `qwen2.5:0.5b` at the shipped sampling settings, giving 72 answers. (These runs pre-date the switch to Qwen 2.5 1.5B and have not been re-run; the personality discriminability result is not expected to change since the behaviour is driven by prompt design, not model scale.) A judge from a *different* model family, `llama3.1:8b`, then reads each answer blind and names the guide that wrote it, choosing from paraphrased descriptions rather than the prompt text, with the options shuffled per item.

Table 8: Blind identification of the guide from its answer. Rows are the guide that wrote the answer, columns the judge’s choice; the diagonal is correct.

true \ predicted	artistic	technical	child
artistic	14	10	0
technical	8	13	3
child	12	11	1

The guides are not distinguishable. 28 of 72 correct, 38.9 %, against a 33.3 % chance baseline — an exact binomial $p = 0.19$, indistinguishable from guessing. The matrix shows where it breaks down: the judge chose *child* four times in 72, so the child guide is recovered in 4 % of its own answers. Reading the output agrees with the number. Asked why Casa Batlló’s facade resembles bones, the child guide replies that it “gives it a sense of solidity and strength”, which is not a sentence written for an eight-year-old.

This is not an artefact of the judge. Given three hand-written, deliberately distinct texts, the same judge scores 3/3 and picks *child* correctly — the label it almost never chose on real output. The instrument discriminates; the guides simply sound alike. This self-test runs on every invocation so the null always ships with evidence the measurement works.

What it cannot say. Without a positive control arm — a deliberately loud persona run under the same constraints — this result cannot separate two causes with opposite remedies: that the three prompts are too similar to each other, or that the regime suppresses any persona at all. Both are plausible. Each prompt is roughly 58 words of which about 28 are verbatim-identical across all three, so only some 30 words distinguish a guide; and one of the shared sentences (“answer ONLY what the user asks — do not give unsolicited background”) forbids precisely the elaboration in which register lives, under a 50-word instruction and a 60-token cap that truncated 29 % to 38 % of answers mid-sentence.

A larger finding: grounding. The same judge was asked, for each answer, whether every claim was supported by the facts the model had been given — reference-free, so that style is not scored as inaccuracy.

Table 9: Answers containing at least one claim unsupported by the retrieved facts.

Guide	Unsupported	Rate
artistic	11/24	46 %
technical	13/24	54 %
child	15/24	62 %

Between a half and two thirds of answers contain something the sheets do not support. Spot-checks confirm the judge is not being harsh: the model states that the Hypostyle Room was built “to house a collection of stone columns” when the sheet describes a covered market. On the evidence here this matters considerably more than which of the three voices delivers it, and it is the open item this benchmark would pursue next.

Recommendation. Small models follow format constraints far more reliably than tone instructions. Replacing “you speak with passion and use evocative metaphors” with a concrete output requirement per guide — name one visual image; cite one number when the facts provide one; include one everyday comparison — is the cheapest change likely to move this result, and it can be re-measured with `uv run python main.py personality`.

Selected. **Qwen 2.5 1.5B Instruct**, Q4_K_M (qwen2.5-1.5b-instruct-q4_k_m.gguf).

Three factors ruled out the 0.5B model. First, **answer coherence**: at median 19.2s on the Cortex-A53 the 0.5B hits the 128-token ceiling on every single answer (all `tokens` fields in the prediction file are exactly 128), producing truncated and often repetitive output; the 1.5B reaches the same limit on far fewer questions and typically closes its sentences within the budget. Second, **personality failure**: the 0.5B does not sustain the guide voice across an answer — register collapses to the same neutral, schematic phrasing regardless of which personality prompt is active, a behavior that makes the three guide characters indistinguishable in practice (section 3.1.4). Third, **shallow abstraction**: the 0.5B model tends to copy verbatim fragments from the retrieved passage rather than integrating them into a coherent explanation; answers are structurally correct but intellectually superficial, which is particularly visible on architect-level questions that require reasoning across multiple facts.

3.2 Speech-to-text

3.2.1 Engine Evaluation

To find the optimal trade-off between keyword retention, memory footprint, and inference latency on the Cortex-A53, several speech-to-text architectures were benchmarked: **faster-whisper** (CTranslate2), **whisper.cpp** (GGML), **sherpa-onnx**, and **vosk**. For the Whisper-based engines, both `tiny.en` and `base.en` tiers were evaluated. The English-only (`.en`) variants were deliberately chosen since the guide answers in English, and monolingual weights are both smaller and more accurate than their multilingual counterparts at the same parameter count.

3.2.2 Metrics and domain biasing

The harness measures Word Error Rate (WER), Character Error Rate (CER), Keyword-spotting Accuracy (KWA), Real-Time Factor (RTF), and peak RAM.

Keyword-spotting accuracy deserves a special note. It is measured against a curated list of domain aliases (e.g., “La Pedrera” or “catenary arch”). A generic WER number hides the failure that actually matters for an audio guide: a transcript that gets “Gaudí” or “trencadís” wrong will retrieve the wrong knowledge sheet, and the whole answer is then confidently off-topic.

To prevent this, the decoder is biased towards the domain vocabulary by injecting the following literal string as the `initial_prompt`:

"Cultura Viva audio guide in Barcelona about Antoni Gaudí, Sagrada Família basilica, Nativity, Passion, and Glory facades, Catalan modernisme architecture, Casa Batlló, Casa Milà, Park Güell, dragon and salamander sculptures, and trencadís mosaics."

3.2.3 Results and selection

The evaluation set consisted of **24 synthetic visitor utterances**, varied across register (neutral, polite, child-like), intent (materials, history), and speaker voice, with controlled background noise.

Table 10: STT benchmark results over the 24-utterance domain set on the Cortex-A53 board.

Engine (Model)	WER	CER	KWA	RTF	Peak RAM	Size
faster-whisper (base.en, INT8)	16.4 %	9.3 %	96.1 %	1.05	397 MB	141 MB
whisper.cpp (base.en, q5_1)	21.1 %	12.5 %	94.7 %	1.08	258 MB	57 MB
faster-whisper (tiny.en, INT8)	32.5 %	18.9 %	79.0 %	0.58	330 MB	75 MB
sherpa-onnx	35.9 %	19.5 %	50.0 %	0.07	464 MB	207 MB
vosk	69.8 %	41.0 %	21.1 %	1.16	395 MB	68 MB

The benchmark clearly highlights the architectural trade-offs. While **sherpa-onnx** is exceptionally fast (RTF 0.07), its keyword accuracy (50%) makes it unusable for a domain-specific

guide. Stepping down to a `tiny.en` model degrades keyword spotting unacceptably to 79 %. The `base.en` tier is the smallest Whisper model that holds up on full sentences.

What ships. `faster-whisper` with the `base.en` model at INT8 (`models/stt/faster-whisper-base.en`). A 16.4 % word error rate alongside a 96.1 % keyword accuracy is exactly the shape we were optimising for: the transcript is imperfect in its function words but almost never wrong about the proper nouns that drive retrieval.

To maximise performance on the board, the shipped engine is configured with:

- **Explicit hotwords** — passing the domain aliases directly to the CTranslate2 decoder.
- **4 CPU threads** — the QRB2210 has exactly 4 Cortex-A53 cores; matching this saturates the CPU without context-switch overhead.
- **INT8 compute type** — keeps the model well inside the LPDDR4 memory and maximises memory bandwidth, which is the binding constraint here.
- **Greedy decoding** — `temperature = 0.0` and `beam_size = 1` to minimise RTF.

3.3 Text-to-speech

3.3.1 Candidates

18 ONNX voices are registered in `models_config.json`: three VITS models exported from Hugging Face (MMS-TTS English, VITS LJSpeech, VITS VCTK) and 15 Piper voices — Lessac, Ryan, Amy, Alan and Cori, each in low, medium and high quality. 15 of the 18 were actually timed; `piper-amy-high`, `piper-alan-high` and `piper-cori-low` were not run.

The field narrowed to Piper, on the inference-time / quality trade-off. The VITS exports were included as the obvious general-purpose baseline and were eliminated by the numbers below: at $\text{RTF} \approx 3.8$ they are far too slow for a device that has to answer while the visitor is standing in front of the building, and the naturalness they return for that cost is not better than a Piper medium voice. Piper, built for exactly this class of hardware, reaches acceptable voice quality at RTF well under 1 — and, crucially, is tiered, so quality can be traded against compute by swapping a voice file rather than changing the runtime. That tiering is what the rest of this section measures.

Audio quality is not scored here — the benchmark measures *inference time and footprint only*. Which specific Piper voice suits which guide personality was decided separately, by ear.

3.3.2 Evaluation Protocol

All benchmarks were executed directly on the Arduino UNO Q hardware (Qualcomm QRB2210, 4x Cortex-A53 threads, 3.6 GB RAM). To ensure realistic and reproducible measurements, each model was loaded into memory and given one untimed warm-up inference. The performance was then timed over **9 evaluation runs** per model, distributed evenly across three sentence-length buckets: short (≤ 8 words), medium (9–16 words), and long (≥ 17 words), all drawn from the domain-specific Gaudí dataset.

3.3.3 Reading the results

Table 11 separates the field cleanly into three groups.

- **The VITS exports are unusable here.** $\text{RTF} \approx 3.8$ means roughly four seconds of compute per second of speech. A 20-second answer would take over a minute to synthesise.

Table 11: TTS inference time on the Arduino UNO Q. Median seconds per sentence by length bucket; RTF is the median of (inference time / generated audio duration), so $RTF < 1$ means the voice synthesises faster than real time. Disk size is the configured on-disk model size.

Voice	Disk (MB)	Short (s)	Medium (s)	Long (s)	RTF
<i>VITS (ONNX Runtime, CPU)</i>					
MMS-TTS (English, VITS, ONNX)	138	12.91	21.95	40.92	3.88
VITS (LJSpeech, ONNX)	332	15.39	43.89	59.04	3.78
VITS (VCTK, multi-speaker, ONNX)	150	13.69	34.43	49.50	3.80
<i>Piper</i>					
Piper (en_US-lessac-low)	20	1.61	4.12	6.00	0.62
Piper (en_US-lessac-medium)	63	1.91	4.91	7.28	0.77
Piper (en_US-lessac-high)	113	12.86	33.82	50.45	5.19
Piper (en_US-ryan-low)	20	1.57	3.89	5.87	0.63
Piper (en_US-ryan-medium)	63	1.94	5.36	7.43	0.80
Piper (en_US-ryan-high)	113	12.52	37.24	51.76	5.31
Piper (en_US-amy-low)	20	1.79	4.28	5.95	0.61
Piper (en_US-amy-medium)	63	2.44	5.83	8.23	0.80
Piper (en_GB-alan-low)	20	1.85	5.07	7.30	0.64
Piper (en_GB-alan-medium)	63	2.39	6.14	8.67	0.75
Piper (en_GB-cori-medium)	63	2.27	4.51	7.19	0.81
Piper (en_GB-cori-high)	113	13.31	31.94	52.32	5.27

- **Piper “high” voices are equally unusable**, and for the same reason — $RTF \approx 5.2\text{--}5.3$, worse than the VITS models despite Piper’s much lighter runtime. The quality tier, not the framework, is what blows the budget.
- **Piper low and medium clear the bar with room to spare**. Every low voice lands at $RTF \approx 0.61\text{--}0.64$ and every medium voice at $\approx 0.75\text{--}0.81$. A medium voice synthesises a short sentence in about 2s and a long one in about 8s, while producing more audio than that in playback time — which is what makes sentence-level streaming (section 3.4) work: playback of sentence N outlasts synthesis of sentence $N + 1$, so the audio never stalls.

The gap between low and medium is around 25 % of compute for a clearly better voice, which is worth paying given the headroom.

Selected. Two Piper medium voices ship on the device, mapped to guide personalities:

Personality	Style	Voice
A — Artistic	Evocative, metaphorical	en_US-libritts_r-medium
B — Technical	Architectural, precise	en_GB-semaine-medium (Spike)
C — Child	Simple, fun, story-led	en_GB-semaine-medium (Prudence)

3.4 End-to-end latency

Adding up every pipeline stage measured in isolation gives an unrealistic upper bound, as the device does not run them strictly sequentially. To evaluate real-world user experience, the benchmark reports **Time to First Audio (TTFA)**: the delay from the moment the visitor stops speaking to the first generated word.

Three mechanisms heavily optimise this perceived latency by overlapping operations:

- **Asynchronous prefill.** The static prompt prefix (personality and knowledge facts) is prefilled into the KV-cache on a background thread while the visitor is still speaking, leaving only the question to be processed at the end.

- **Silence cropping.** Recording automatically trims silent margins before passing the audio to Whisper, preventing the encoder from wasting compute on padded room tone.
- **Sentence-level streaming.** Rather than waiting for the full SLM output, tokens are emitted per sentence. Piper synthesises sentence N while the SLM decodes sentence $N + 1$, ensuring audio playback never stalls.

Together, these concurrent techniques transform the TTFA from a naive sequential sum of all stages into a highly optimised pipeline:

$$\text{TTFA} \approx \text{STT} + \text{warm prefill} + \text{decode}_{n=1} + \text{TTS}_{n=1}$$

4 Bill of materials

4.1 Arduino components

Prices are Arduino Store (EU) list prices, VAT included, retrieved from <https://store.arduino.cc>.

Table 12: Arduino components. The three Modulino modules are the ones this project requires.

Component	Code	Qty	Role in Cultura Viva	Price
Arduino UNO Q 4GB	—	1	MCU + Linux MPU, the whole system	82.90 €
<i>Modulino modules (I²C, Qwiic / Wire1)</i>				
Modulino Buttons	ABX00110	1	Guide personality select A/B/C + LEDs	8.50 €
Modulino Knob	ABX00107	1	Live headphone volume 0–100 %	9.00 €
Modulino Buzzer	ABX00108	1	Shutter click, confirmations, alerts	7.30 €
Modulino subtotal				24.80 €
Arduino subtotal				107.70 €

4.2 Third-party components

Due to the board’s form factor and the AI pipeline’s peak power demands, two specific external components are strictly required: a **USB-C hub** with Power Delivery (PD) pass-through (to connect the camera, headphones, and power to the single USB-C port), and a **high-output power bank** (≥ 15 W sustained) to prevent brownouts during full-core inference.

4.3 Software cost

Zero. Every model in the pipeline is open-weights and runs locally, so there are no inference API calls and no per-visitor marginal cost. The Ragas judge used during evaluation (**qwen2.5:7b** via Ollama) is also local, and is a development-time dependency that never ships to the device.

5 Summary of selections

Table 13: Non-Arduino components. *These prices were not verified against a named vendor — replace with your actual purchase prices before publishing.*

Component	Interface	Qty	Role	Price
USB-C hub, PD pass-through	USB-C	1	Camera + headphones + power on the board’s single USB-C port	19.99 €
Power bank, USB-C PD ≥ 15 W, $\geq 10\,000$ mA h	USB-C	1	Battery operation at the high-performance profile	55.00 €
ST7735S LCD, 1.8" 128×160	SPI	1	Live view, minimap, UI	7.99 €
Logitech Brio 105 webcam	USB 2.0	1	1080p capture + microphone	21.00 €
NEO-6M GPS module	Serial1 @ 9600	1	Selects the active minimap	10.84 €
Mode switch (SPDT)	Digital D6	1	Camera / map mode	1.25 €
Push button	Digital D7	1	Shutter / record / cancel	1.00 €
Headphones or speaker	3.5 mm jack	1	Spoken answer output	14.90 €
Jumper wires, carrier board	—	—	Assembly	10.55 €
Third-party subtotal				142.52 €

Table 14: What the benchmarks selected, and on what grounds.

Stage	Selected	Decided on
Vision	Per-monument ONNX classifiers, 224×224 , with entropy + confidence OOD gate	Accuracy / macro-F1 per monument; OOD rejection is a hard requirement, not a nicety.
STT	faster-whisper <code>base.en</code> , INT8, 4 threads, greedy	16.4 % WER but 96.1 % keyword accuracy — proper nouns drive retrieval.
SLM	Qwen 2.5 1.5B Instruct Q4_K_M	Best answer coherence within the 128-token cap at 53 s median on the Cortex-A53; grounding comparable to 0.5B (33.3 % vs. 31.9 % unsupported), pipeline validated at 1.45 GB working set.
TTS	Piper medium voices, per personality	RTF ≈ 0.8 on the board, which is what makes sentence-level streaming viable.

6 Interactive Web Demo

While the core engineering achievement of *Cultura Viva* is its fully on-device execution on the Arduino UNO Q, we understand that reviewers and users may not have immediate access to this specific hardware to test the pipeline.

To bridge this gap and allow evaluators to easily experience the system’s conversational flow, vision-triggered context, and overall interaction model, we have deployed a cloud-hosted web version of the project. This interactive interface replicates the physical device’s AI capabilities within a standard browser environment.

The live demonstration can be accessed at <https://demo.culturaviva.tech> using the following credentials:

- **Username:** culturaviva
- **Password:** gaudi2026

TTS inference time by sentence length, vs. model size

Arduino UNO Q (Qualcomm QRB2210, Cortex-A53, Linux) · median $\pm$ min/max per length bucket, over Gaudí-related sentences

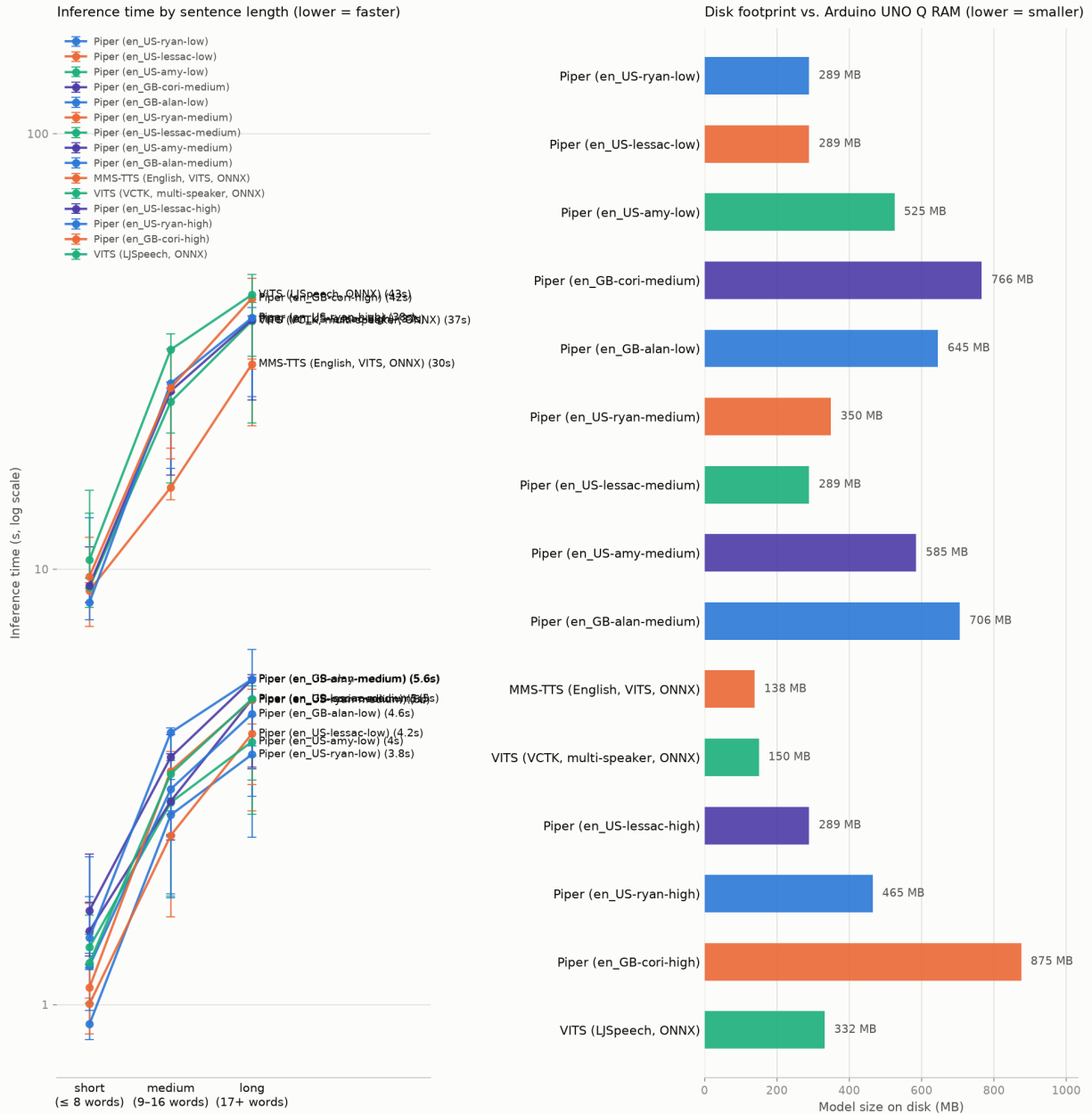


Figure 2: Median inference time with min/max whiskers per sentence-length bucket, alongside on-disk model size, generated by `visualize.py` from the same results file as table 11.